INDEPENDENT PORTFOLIO | OCTOBER 2026

TableTap restaurant service workflow

Working prototype, employee discovery, and implementation roadmap

Zachary Fogle | Prototype development, stakeholder interviews, and workflow analysis

View the application source

Decision in brief

Advance TableTap to a structured employee walkthrough before considering live restaurant use. The prototype captures table-specific requests, persists them locally, and lets authenticated staff claim or reassign ownership. The next design challenge is operational: confirm how the table server stays informed, define queue coverage, and determine what happens when nobody can monitor requests.

The central finding is that visibility and accountability are separate problems. A digital request can make a guest need visible, but the operating model must still assign responsibility and confirm fulfillment.

Evidence available	What it establishes
Working prototype and local verification	Table routing, submission, shared display, status changes, completion, and reopening.
Two California Dreaming server interviews — July 27, 2026	Employee needs and adoption risks that shape the proposed operating model.
Ten passing automated tests	Validation, delivery, persistence, restaurant isolation, ownership history, and stale-update behavior.

The service problem

Restaurant service experience showed that routine needs such as napkins, extra plates, or the check may remain unknown while a server is helping another table. Interviews with Hannah and Savannah, two California Dreaming servers, reinforced the importance of preserving personal service while making routine requests easier to surface.

The business question is whether an optional digital channel can improve request visibility without creating another screen to monitor, duplicate work, or uncertainty about who should respond.

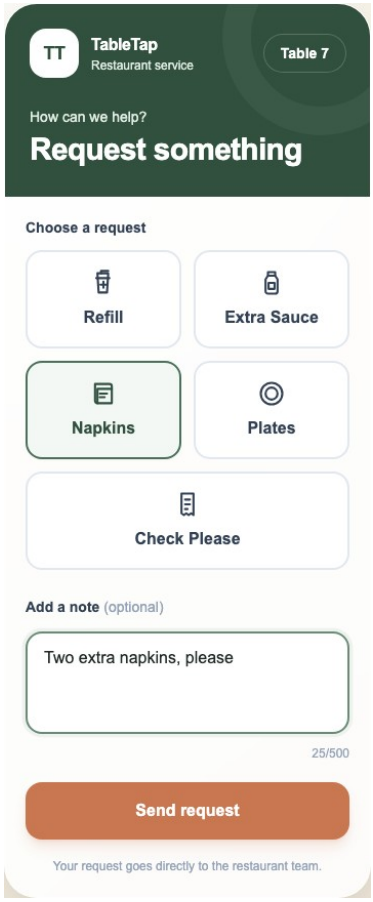
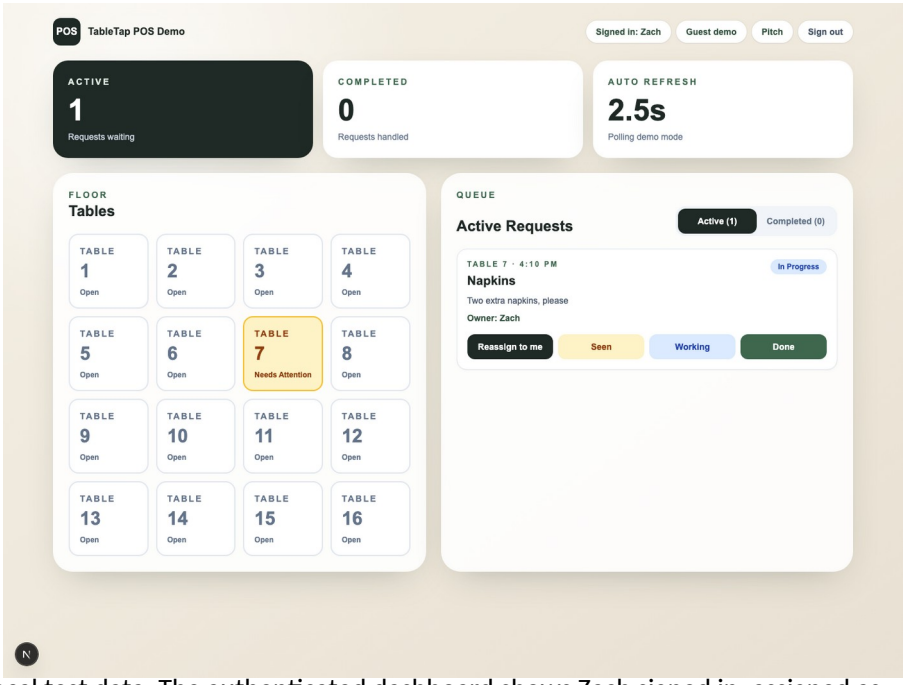
What the workflow changes

Stage	Current service	Proposed TableTap workflow
Capture	Guest gets an employee's attention.	Guest opens the table QR page and submits a request.
Coordinate	Employee fulfills or asks a colleague.	Staff member claims the request; ownership is visible in the shared queue.
Close	Verbal confirmation or table check.	Responder confirms delivery and closes the request.

Digital intake, persistent status history, authenticated staff assignment, and restaurant-scoped queue access work in the prototype. Server notification, queue coverage, and capacity controls are the next operating requirements to validate.

From table QR to staff queue

One dynamic route, /table/[tableId], carries table context into the guest page. Five quick-request buttons and an optional note support routine needs; a note alone becomes a Custom Request. A validated API sends the request to the built-in demo queue and any configured delivery adapters.

Guest view at Table 7	Staff dashboard during the same local demonstration
	

Local test data. The authenticated dashboard shows Zach signed in, assigned as owner of Table 7's napkin request, and the request marked In Progress.

Observed path: submit → New → Seen → In Progress → Done → Completed tab → Reopen. The browser check verified each step. Marking Done records software state; it does not independently verify physical delivery.

Implementation choices and tradeoffs

Next.js, React, and TypeScript provide the guest pages, staff interface, and API in one application. Tailwind CSS handles styling. The dashboard polls every 2.5 seconds and includes table-level views, active/completed tabs, and four request states.

A local SQLite database preserves requests, ownership, completion timestamps, and event history across restarts. Signed eight-hour staff sessions protect queue access, restaurant IDs scope data, and version checks reject updates from stale queue views. Shared deployment still requires managed identity and database services.

Guest screenshot captured September 22, 2026; authenticated staff dashboard captured October 16, 2026. Persistence, ownership, access control, and restaurant scoping were verified locally. The demonstrations used local test data.

What the interviews change

The interviews changed the design question from “Can a guest send a request?” to “How does the restaurant respond without weakening service?” Their feedback points to four requirements: optional use, clear ownership, server awareness, and a workflow simple enough for busy shifts.

Evidence	Design implication	Current status and next action
Interview synthesis: routine needs can be difficult to surface while staff are occupied.	Give guests an optional way to communicate a routine request.	Table-specific request intake works in the prototype.
Interview synthesis: technology should support personal service rather than replace it.	Keep verbal requests and normal table check-ins.	Clarify the guest-facing language and preserve a staff fallback.
Interview synthesis: unclear responsibility can lead to missed or duplicate work.	Separate request visibility from employee ownership.	Claim and reassignment controls use authenticated staff identity and stale-update protection.
Interview synthesis: any new workflow must remain usable during busy periods.	Minimize staff steps and make queue coverage explicit.	Test task time, unclaimed requests, and queue-monitoring effort.
Research boundary: two server perspectives inform the design but do not establish approval.	Seek manager and additional staff review before a pilot.	Use a structured walkthrough as the next decision gate.

Interview scope and decision boundary

The two interviews provide direct employee perspective, but they do not represent a restaurant-wide decision. A manager and additional staff roles should review the proposed workflow before any live evaluation.

Evaluate the alternatives

Option	What it addresses	Decision now
Existing service	Maintains the familiar guest interaction.	Retain as the operating baseline.
Clearer verbal handoffs	Could clarify responsibility after staff hear a request.	Explore as a low-complexity process change; it does not remove the attention gap.
QR intake with a shared staff queue	Surfaces routine needs without requiring the guest to first find an employee.	Advance to an employee walkthrough; validate ownership, workload, and acceptance.

Method: separate interviews with Hannah and Savannah, both California Dreaming servers, on July 27, 2026. Findings are summarized as themes rather than presented as verbatim quotations. The sample is useful for design decisions, not for estimating adoption across employees or guests.

Evidence, gaps, and next steps

Technical verification

Local verification confirmed the guest-to-dashboard flow, staff login, unauthorized API rejection, restaurant-scoped access, claim and reassignment controls, event history, and persistence across an application restart. All ten tests passed, lint passed, and a production build passed with Webpack.

The tests cover request validation and delivery behavior plus ownership persistence, restaurant isolation, status history, and stale-update conflicts. External services are mocked. These checks establish selected technical behavior; employee usability and restaurant service outcomes remain the next evidence gap.

Requirements before restaurant use

Priority	Gap in the current prototype	Required decision or change
Before live use	The shared demonstration PIN does not provide managed individual employee accounts.	Replace the shared PIN with managed staff identity before live use.
Before live use	Local SQLite storage is not shared across serverless application instances.	Move the repository to a managed database before shared deployment.
Before live use	No queue-coverage state, intake pause, or staff escalation policy exists.	Add capacity controls and validate who monitors requests each shift.
Before live use	No actual Toast/KDS connection or venue-approved table mapping exists.	Choose an approved staff display route before any venue integration.

Next decision gate

Proposed next step: run a 15-minute walkthrough with a server and shift leader. Follow one routine request through submission, assignment, delivery, and acknowledgement; then introduce a duplicate request and a period when no employee can monitor the queue. Record confusion, missing controls, and workarounds before deciding whether a limited pilot is warranted.

Measurement if a live test is authorized

If a live test is approved, log request receipt, assignment, actual delivery, duplicate handling, staff effort, and unresolved items. Compare similar shifts and request types, including verbal requests, so the analysis measures the complete service workflow rather than only digital activity.

The prototype now records creation, assignment, status changes, and completion in a durable local event history. A live evaluation would still need consistent definitions for actual delivery time and staff effort.

Outcome so far

TableTap now demonstrates table-specific request capture, persistent history, authenticated access, restaurant scoping, and explicit staff ownership. The employee interviews shaped those changes and the operating recommendation: preserve personal service, keep ownership visible, and validate workload before expansion. The next milestone is an employee walkthrough with defined success and stop criteria.